

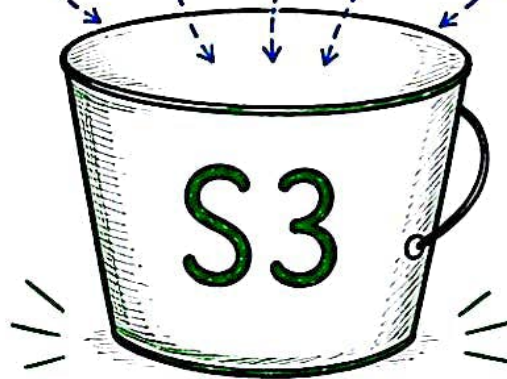


S3

Simple Storage Service

Where do millions of photos, videos, documents and backups live on the cloud?

Behind the scenes, it's Amazon S3.



In this series, we will learn:

- | | |
|--|--|
| ☒ Core concepts of Amazon S3 | ☒ Advanced features & Best practices |
| ☒ Security, Permissions & Policies | ☒ Real-world architectures |
| ☒ Storage classes & Lifecycle | ☒ Troubleshooting & Interview concepts |
| ☒ Pricing & Cost optimization | |

= ★ Master S3. Store anything. Access anywhere. ☁



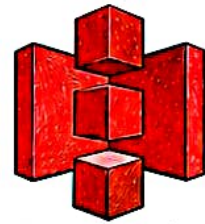
WHAT IS AMAZON S3?

2

Amazon S3 (Simple Storage Service) is an object storage service offered by AWS.

It allows you to store and retrieve any amount of data from anywhere on the internet.

S3 is designed for 99.999999999% (11 nines) durability, high availability, security and scalability.



Amazon S3
Simple Storage Service

Why Do We Need S3?

- ✓ Store unlimited data without worrying about infrastructure.
- ✓ Access data from anywhere, at any time.
- ✓ Highly durable and available.
- ✓ Pay only for what you use.
- ✓ Scales automatically as your data grows.

Traditional Storage vs S3

Traditional Storage



- ✗ Limited capacity
- ✗ High maintenance
- ✗ Expensive
- ✗ Not easy to scale

VS

Amazon S3



- ✓ Virtually unlimited
- ✓ No maintenance
- ✓ Cost-effective
- ✓ Scales automatically

Simple Analogy



- Think of S3 like a **bucket**.
- In this bucket, you can keep any number of items (files). These items are called **objects**.
- Each object has a name (**key**) and some information (**metadata**).
- You can add, remove or access these objects from anywhere.

Key Takeaways

- S3 = Simple Storage Service → Object Storage on AWS.
- Store any type of data (images, videos, documents, backups, logs, etc.).
- Built for durability, availability, security and scalability.
- Used by startups, developers and enterprises around the world!

If you have data,
S3 is the perfect place
to store it!



Millions of applications rely on Amazon S3 to store and protect their data.
It is the foundation of modern cloud applications.





S3 BUCKET & OBJECT

— THE FOUNDATION OF S3 —



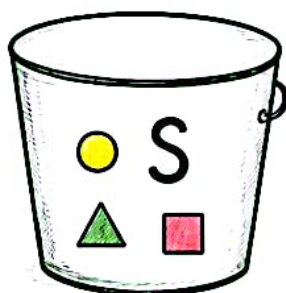
3



In S3, everything is stored inside a BUCKET.
Inside the bucket, data is stored as OBJECTS.

1. What is a Bucket?

- A bucket is like a container in S3.
- It must have a globally unique name.
- All objects are stored inside a bucket.
- Bucket name is visible in the Amazon S3 URL and in the AWS account.



Bucket Name
my-bucket-123

2. What is an Object?

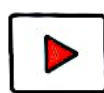
- An object is the basic entity stored in S3.
- It can be a file (photo, video, document, log file, backup, etc.).
- Each object consists of the data, a key (name) and metadata.
- Objects can have different sizes, from a few bytes to several TBs.



photo.jpg



report.pdf



video.mp4



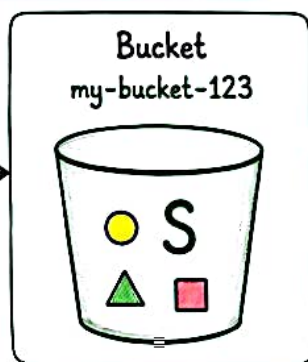
app.log

All these are objects inside a bucket.

3. Relationship: Bucket contains Objects



Your AWS
Account



Objects inside the bucket



photos/photo1.jpg



documents/report.pdf



videos/tutorial.mp4



logs/app.log

...

4. Real-World Example

Real World	S3 Concept
 A Building	Your AWS Account 
 Rooms in the Building	Buckets in S3 
 Items in the Rooms (Chairs, Tables, Files, etc.)	Objects in the Bucket    



Key Points

- ✓ You can create multiple buckets.
- ✓ Each bucket holds unlimited objects.
- ✓ Objects in a bucket are flat (not in folders, but keys can look like folders).
- ✓ Bucket name is unique across the entire AWS globe.



Bucket is the container. Object is the data. Together they make **Amazon S3** powerful!



S3 OBJECT KEY & METADATA

4



Every object in S3 is uniquely identified by its **KEY** and has additional information called **METADATA**.



1. What is Object Key?

- Object key is the unique name (or path) of an object inside a bucket.
- It includes the object name and optional folder (prefix) structure.
- Keys are **case-sensitive**.
- You cannot have two objects with the same key in the same bucket.

Example Keys

- images/photo1.jpg
- documents/reports/2024/report.pdf
- videos/tutorials/aws/ec2-demo.mp4
- logs/app1/2024/05/20/error.log

3. What is Metadata?

- Metadata is data about the object.
- It provides important information like type, size, time, owner, etc.
- Metadata helps applications understand and manage the object better.

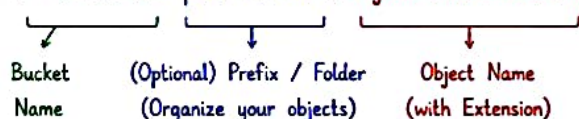
4. Types of Metadata

System-Defined Metadata (Set by S3 Automatically)	User-Defined Metadata (Set by You)
- Content-Type (MIME type) - Content-Length (Size) - Last-Modified - ETag - Storage-Class - Cache-Control - Server-Side Encryption info	- Custom key-value pairs - Example: - x-amz-meta-author: satish - x-amz-meta-project: cloudvault - x-amz-meta-language: python - x-amz-meta-env: production - You can add your own meaning to metadata.

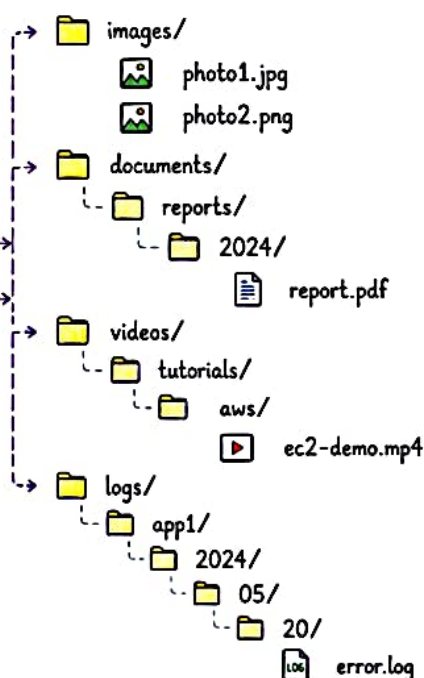
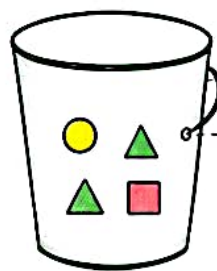
2. Key Structure

General Format:

s3://bucket-name/prefix/folder/object-name.extension



S3 Bucket
(my-bucket)



5. Example: Object with Metadata

Object Key:

documents/reports/2024/report.pdf

Sample Metadata

Metadata Key	Value
Content-Type	application/pdf
Content-Length	204857
Last-Modified	2024-05-20 10:30:00 UTC
ETag	"8f3a2b...9d7c"
x-amz-meta-author	satish
x-amz-meta-project	cloudvault
x-amz-meta-env	production



Key Takeaway

Key identifies the object. Metadata describes the object.

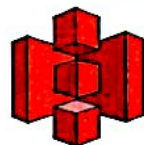
Both are used together to store, find and manage data in S3 efficiently.





S3 HOW TO CREATE A BUCKET & UPLOAD AN OBJECT

5



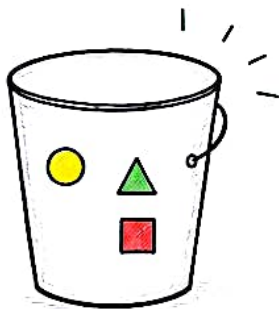
S3



Let's see the basic steps to create a Bucket and upload an Object in S3.

1 CREATE A BUCKET

- Go to S3 Console.
- Click on "Create bucket".
- Provide a unique name to your bucket.
- Select the AWS Region where you want to store the data.
- Leave other settings as default for now.



Bucket Naming Rules

- 3 - 63 characters
- Lowercase letters, numbers, dots (.) and hyphens (-)
- Must start and end with a letter or number
- No two adjacent dots (..)
- Must be globally unique

2 CONFIGURE BASIC SETTINGS

- Bucket name : my-bucket-123
- AWS Region : ap-south-1 (Mumbai)
- Object Ownership : ACLs disabled (recommended)
- Block Public Access : ON (recommended) (keeps your data private)



For beginners, keep all other settings as default. You can change later when required.

3 CREATE THE BUCKET

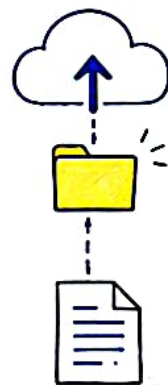
- Review the details.
- Click on "Create bucket".
- If the name is available, you will see success message.



Your bucket is now ready!
You can start uploading objects.

4 UPLOAD AN OBJECT

- Open your bucket.
- Click on "Upload".
- Click "Add files" and select a file from your computer.
- (Optional) Add a prefix / folder like images/, docs/report.pdf
- Click "Upload" to start uploading.



5 VERIFY THE OBJECT

- After upload, your file (object) will appear in the bucket.
- You can view file details like Size, Type, Last modified, Storage class.
- Now your data is safely stored in S3!

Example Bucket: my-bucket-123

Name	Type	Last modified
images/	Folder	-
report.pdf	pdf	20 May 2024 10:30 AM
photo.jpg	jpg	20 May 2024 10:31 AM



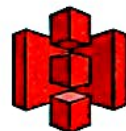
KEY TAKEAWAY

Bucket is a container, Object is the file. With S3, you can store any amount of data safely, durably and cost-effectively.





S3 VERSIONING



6

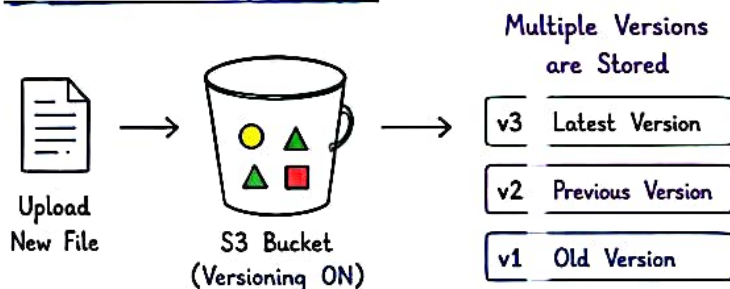


S3 Versioning keeps multiple versions of an object in the same bucket. It helps to protect your data from accidental deletion or overwrite.

1. Why Versioning?

- Protect against accidental delete or overwrite.
- Recover old versions easily.
- Helps in auditing and compliance.
- No extra charge for storing versions (you pay only for storage used).

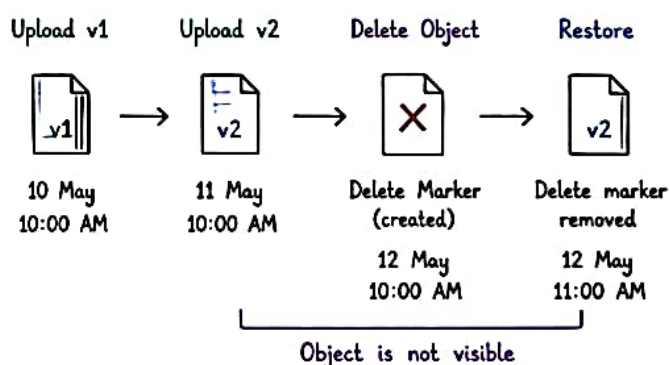
2. How Versioning Works?



Every time you upload an object with the same key, S3 creates a new version instead of replacing the old one.

3. Delete in Versioned Bucket

- When you delete an object, the **actual data** is not deleted.
- Instead, a Delete Marker is created.
- You can remove the delete marker to restore the object.



4. Enable Versioning (Steps)

- ① Open your S3 Bucket.
- ② Go to Properties tab.
- ③ Scroll down to Versioning.
- ④ Click **Edit**.
- ⑤ Select Enable versioning.
- ⑥ Click Save changes.

Versioning

Keep multiple versions of an object in the same bucket.

[Learn more](#)

Versioning

- ☐ Suspend
☒ Enable

Cancel

Save changes

5. Point to Remember

- ✓ Versioning is bucket-level, not object-level.
- ✓ Once enabled, it **cannot be disabled**, you can only suspend it.
- ✓ **Suspended** = new versions are not created, but old versions are kept.

- ✗ Versioning does not protect against **misconfigured policies** or access.
- ✗ Delete marker is a placeholder, it is not the actual file.



KEY TAKEAWAY

Versioning is like an "undo" for your data. It keeps older versions safe, so you can recover anytime you need.



- ✓ Accidentally deleted? - Recover
- ✓ File overwritten? - Rollback
- ✓ Need old data? - Restore

S3 LIFECYCLE RULES

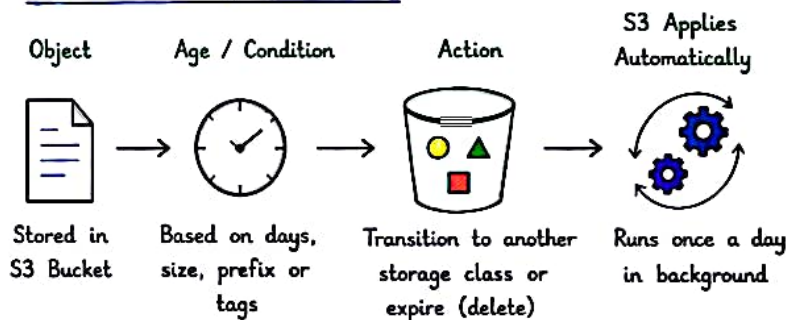
7

S3 Lifecycle Rules help you automatically manage the lifecycle of objects in your bucket based on time, size, or tags. You can transition objects to cheaper storage classes or **expire (delete)** them after a defined period to reduce cost and keep your bucket organized.

1. What are Lifecycle Rules?

- They are bucket-level rules that apply to objects.
- Rules run automatically once a day.
- You can use time, size, or tags as conditions.
- They help you save storage cost.
- No manual work is needed.

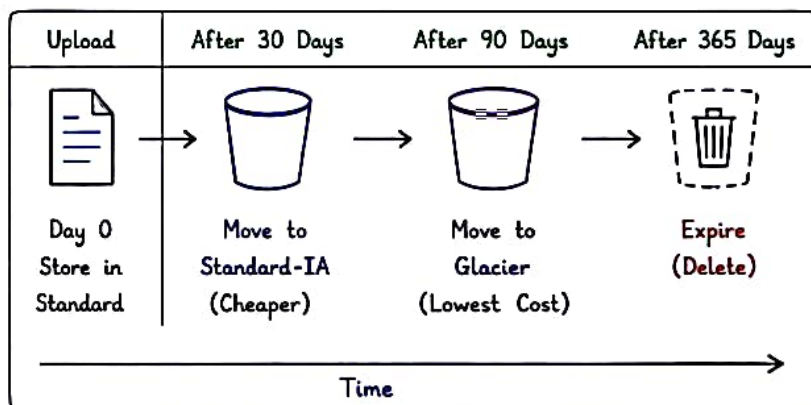
2. How Lifecycle Works?



Note: Once the object matches the rule conditions, S3 automatically takes the defined action. You don't need to run anything manually.

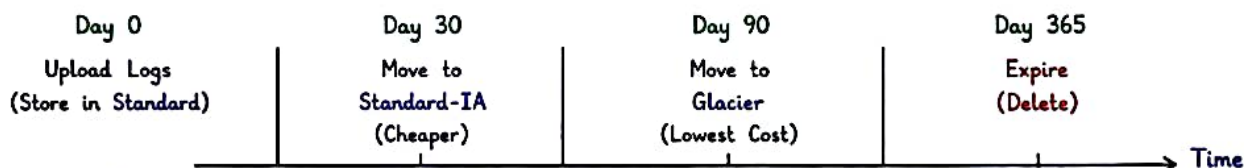
3. Lifecycle Actions

- Transition: Move objects to another storage class when they are not frequently accessed to reduce cost.
- Expiration: Permanently delete objects after a specific time.
- Delete Old Versions: If versioning is enabled, you can delete previous versions after a defined number of days to save space.



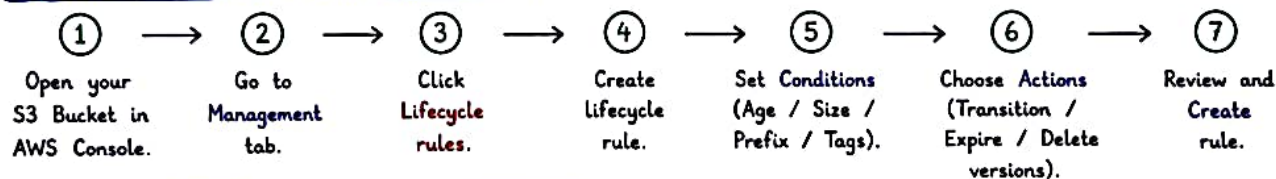
4. Example: Lifecycle Rule for Logs Folder

Scenario: You have logs in S3 that are not needed after some time.



You can create different rules for different folders, file types, or tags.
Example: Move *.log files to Glacier after 90 days.

5. How to Configure Lifecycle Rule?



- ✓ Lifecycle rules run automatically once a day.
- ✓ They help reduce storage cost and improve performance.
- ✓ Use transition for old data and **expiration** to clean unnecessary data.
- ✓ Set the right rule = Big cost savings + Organized bucket.





S3 STORAGE CLASSES 8

S3 Storage Classes are different types of storage designed for different use cases. Each class is optimized for a specific access pattern. You can choose the right class to balance cost, performance and data access pattern.

1. What are Storage Classes?

- S3 offers multiple storage classes to store your data.
- Each class is optimized for a specific access pattern.
- You pay less for data that is accessed **less frequently**.
- You can change the class later using Lifecycle Rules.

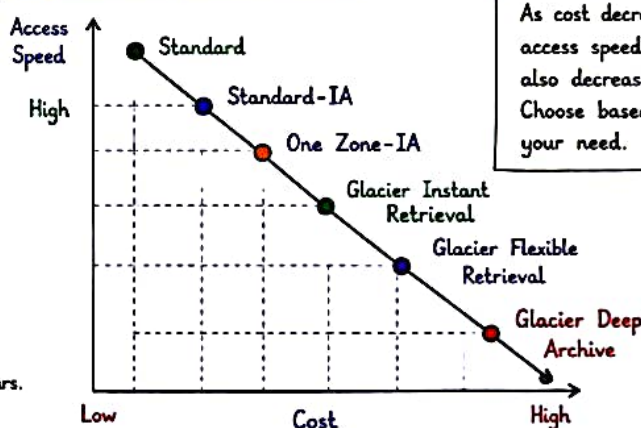
2. S3 Storage Classes Overview

Storage Class	Best For	Availability	Min. Storage Duration	Retrieval Cost
Standard	Frequently accessed data	99.99%	—	Low
Standard-IA	Infrequently accessed data	99.9%	30 days	Low
One Zone-IA	Recreateable, less accessed data	99.5%	30 days	Low
Glacier Instant Retrieval	Long-term data, instant access	99.9%	90 days	Medium
Glacier Flexible Retrieval	Long-term data, minutes to hours access	99.9%	90 days	Low
Glacier Deep Archive	Long-term data, lowest cost	99.9%	180 days	Lowest

3. Storage Classes Explained

- **Standard** : Default class. High durability, low latency, used for frequently accessed data.
- **Standard-IA** : Lower cost, designed for data that is accessed less often.
- **One Zone-IA** : Stores data in a single AZ. Lower cost than Standard-IA.
- **Glacier Instant Retrieval** : Archive data with millisecond access.
- **Glacier Flexible Retrieval** : Archive data with retrieval in minutes or hours.
- **Glacier Deep Archive** : Lowest cost archive, retrieval in hours.

4. Access Speed vs Cost



5. How to Choose the Right Class?



Understand your data access pattern



Choose a storage class based on frequency of access



Use Lifecycle Rules to move data automatically



Save cost and manage data efficiently



You can change the storage class anytime without downtime.

6. Example: Logs Data Strategy

Day 0
Upload Logs



Store in Standard

Day 30
Move to Standard-IA



Lower cost for less access

Day 90
Move to Glacier Flexible Retrieval



Archive with minutes/hours access

Day 365
Move to Glacier Deep Archive



Lowest cost for long retention

After Retention
Expire (Delete)



Data removed automatically

Use Lifecycle Rules to automate this flow and save significant cost over time.



Right storage class = Right cost.
Store smart. Save more. Manage better.





S3 SECURITY

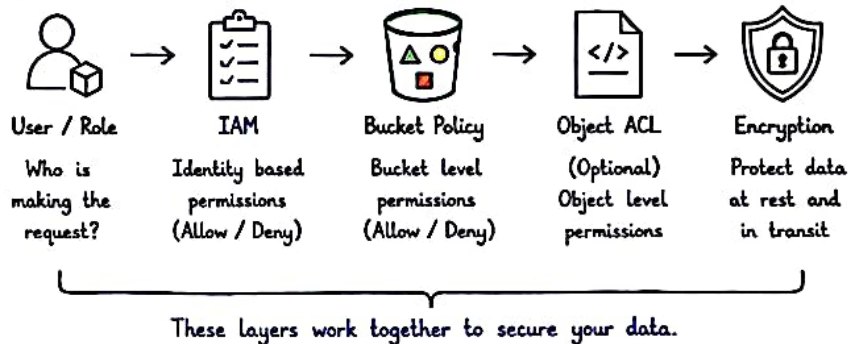
9

Securing your S3 bucket and data is very important. By default, everything in S3 is **private**. You control who can access your data and what actions they can perform.

1. S3 Security Basics

- By default, all buckets and objects are private.
- You decide who can access your data and what actions they can perform.
- You can use multiple security layers together for maximum protection.
- Always follow Least Privilege Principle.

2. S3 Security Layers



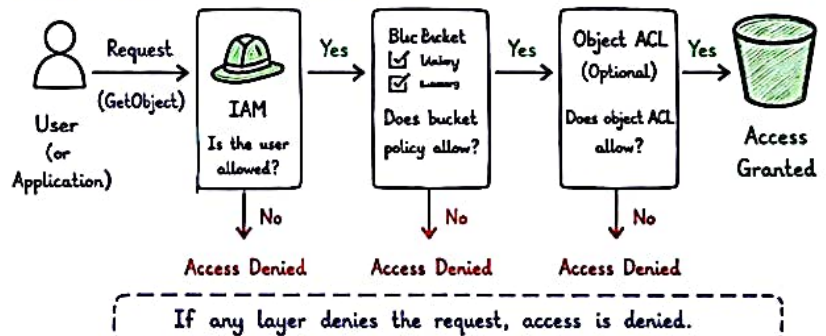
3. Block Public Access (BPA)

Enable Block Public Access at account or bucket level to prevent accidental public exposure.

- ✓ Block public ACLs
- ✓ Ignore public ACLs
- ✓ Block public bucket policies
- ✓ Restrict public bucket policies

Recommended: Keep all enabled

4. Access Control Flow



5. Bucket Policy - Example (Restrict by IP)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowFromOfficeIP",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": "arn:aws:s3::my-bucket/*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

Only requests from IP range 203.0.113.0/24 can access the bucket.

You can also restrict by VPC, AWS Account, or IAM conditions.

6. Secure Data with Encryption

In Transit (Data moving)

Use HTTPS / TLS



Encrypts data while it travels over the internet.

At Rest (Data stored)

Use SSE Encryption

- SSE-S3 (AES-256)
- SSE-KMS (AWS KMS)
- SSE-C (Customer Provided Keys)



★ Enable Default Encryption on the bucket to automatically encrypt all new objects.

7. Common Security Best Practices

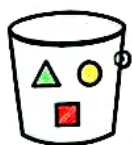
Practice	Why it Matters	How to Implement
Keep Block Public Access ON	Prevents accidental public exposure	Enable BPA at account and bucket level
Use Least Privilege	Reduces risk of unauthorized access	Give only required permissions to IAM users/roles
Use Bucket Policies	Control access at bucket level	Write policies with specific conditions (IP, VPC, etc.)
Enable Encryption	Protects data from unauthorized access	Use SSE-S3 or SSE-KMS
Enable Access Logs	Helps in monitoring and auditing	Enable Server Access Logging to another S3 bucket

8. Remember

- ★ Default: Private
- ★ Use multiple layers for security
- ★ Block Public Access = ON
- ★ Follow Least Privilege
- ★ Encrypt data (in transit & at rest)
- ★ Monitor with Access Logs



Secure today,
Sleep peacefully tomorrow!



S3 IAM vs BUCKET POLICY

10

In S3, access is controlled using IAM Policies and Bucket Policies. Both are JSON based policies, but they work at different levels.

IAM = Who can do what (Identity based) | Bucket Policy = What is allowed on the bucket

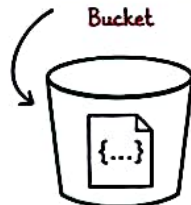
1. IAM Policy (Identity Based Policy)

- Attached to IAM User, Group or Role.
 - Defines what actions the identity is allowed to perform on AWS resources.
 - Permissions allow or deny access.
 - Does NOT grant access by itself.
- Explicit allow = access allowed.
No allow = access denied.

2. Bucket Policy (Resource Based Policy)

- Attached to an S3 bucket.
- Defines who can access the bucket and what actions they can perform.
- Can allow or deny access.
- Can grant access even if IAM policy does not allow (cross-account access).
- Useful for public access, IP restrictions, or cross-account sharing.

Bucket Policy is attached to the Bucket

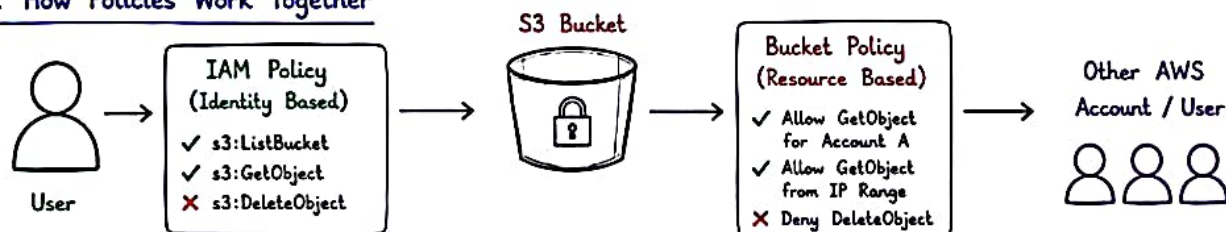


(Resource Level)

3. IAM Policy vs Bucket Policy - Key Differences

Aspect	IAM Policy (Identity Based)	Bucket Policy (Resource Based)
Attached To	IAM User, Group, Role	S3 Bucket
Controls	What the identity can do	Who can access the bucket and what they can do
Scope	Applies to all services (AWS wide)	Applies only to the specific S3 bucket
Effect	Allow or Deny	Allow or Deny
Grants Access?	No (needs resource policy)	Yes, can grant access (even cross-account)
Example Use Case	Give permissions to developers	Allow another AWS account to access bucket
Evaluation	Evaluated with other policies (Union of Allows)	Evaluated with IAM policy (AND logic) Both must allow.

4. How Policies Work Together



Final Access = Allowed only if BOTH policies allow the action.
If any one of the policies denies, access is denied.

5. Example Policies

IAM Policy Example (Allow Read Access)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject"],
      "Resource": [
        "arn:aws:s3::my-bucket",
        "arn:aws:s3::my-bucket/*"
      ]
    }
  ]
}
  
```

Allows listing and reading objects.

Bucket Policy Example (Allow Cross Account)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"AWS": "123456789012"},
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::my-bucket/*"
    }
  ]
}
  
```

Allows another AWS account to read objects.

6. When to Use What?

Use IAM Policy when:

- ✓ You want to give permissions to your users, groups or roles.
- ✓ You want to control what actions they can perform across AWS.
- ✓ For internal access management.



Use Bucket Policy when:

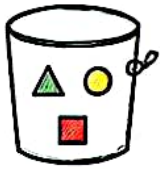
- ✓ You want to share the bucket with another AWS account.
- ✓ You want to allow public access (to website, files etc.)
- ✓ You want to restrict access by IP, VPC or conditions.



★ Golden Rule

Use both together for secure and controlled access.
Least Privilege + Explicit Deny
= Strong Security





S3 PRESIGNED URL

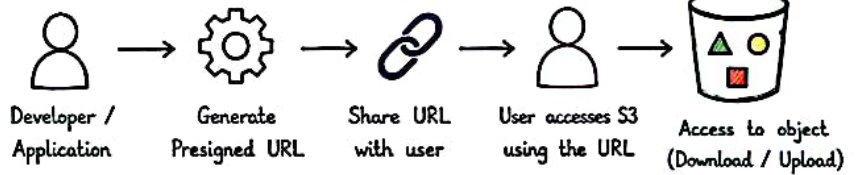
11

A Presigned URL is a temporary, secure URL that gives time-limited access to an S3 object or bucket without exposing AWS credentials. It is commonly used to share files securely or allow direct uploads from users.

1. What is a Presigned URL?

- A Presigned URL is a **special** URL that is signed using AWS credentials.
- It provides limited access to an S3 object or bucket for a specific time period.
- No AWS credentials are required by the user who uses the URL.
- Access is controlled by permissions of the IAM user/role that generated the URL.

2. How It Works



Common Use Cases

- ★ Share files with users outside AWS.
- ★ Allow users to upload files directly to S3 (without giving AWS access).
- ★ Temporary access for downloads (reports, logs, backups, etc.).
- ★ Secure access in web & mobile applications.

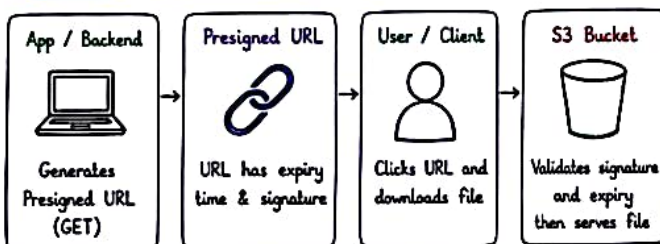
3. Types of Presigned URLs

Type	Purpose	Example
GET (Download)	Allow users to download an object from S3	Share a report.pdf for download
PUT (Upload)	Allow users to upload an object to S3	Let user upload photos directly to S3
DELETE	Allow deletion of an object	Temporary cleanup links
LIST (Bucket)	Allow listing objects in a bucket	Show list of files in a folder

4. Key Properties

- **Time-limited:** You set an expiry time (seconds/minutes/hours).
- **Permission-bound:** Works only if the IAM user/role has permission for that action.
- **Secure:** Uses signature (via AWS Signature Version 4) to validate the request.
- **Resource-specific:** URL is for a specific object or bucket and action.
- **Cannot be revoked easily:** Once generated, it remains valid until expiry.
- **Supports query parameters:** e.g., response-content-disposition (for download filename).

5. Example Flow (Download)



💡 User never gets AWS credentials. Access is controlled by the URL.

6. Example (boto3 - Python)

```
import boto3
from botocore.client import Config

s3 = boto3.client('s3',
    config=Config(signature_version='s3v4'))

url = s3.generate_presigned_url(
    'get_object',
    Params={'Bucket': 'my-bucket',
            'Key': 'reports/report.pdf'},
    ExpiresIn=3600 # 1 hour
)

print(url)
```

← Generates a presigned URL valid for 1 hour

7. Important Notes

- URL expires after the specified time.
- If the object is deleted or permissions are removed, URL will stop working.
- Keep expiry time as short as possible for security.
- Do not share presigned URLs publicly (anyone with the URL can access until expiry).

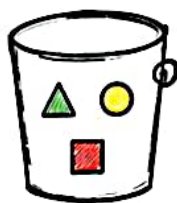
8. Best Practices

- ✓ Use short expiry time (minutes/hours, not days).
- ✓ Generate URLs on the server side, not in the client.
- ✓ Use HTTPS always.
- ✓ Prefer specific actions (GET or PUT) instead of broad access.
- ✓ Validate the object key and user input before generating URL.



Presigned URLs provide **secure**, **temporary** access to S3 objects without sharing AWS credentials. Share safely. Control access. Stay secure.





S3 ENCRYPTION

12

Encryption in S3 protects your data by converting it into unreadable format (cipher text) so that only authorized users can read it.

Encryption can be applied to data at rest (stored in S3) and in transit (moving to or from S3).

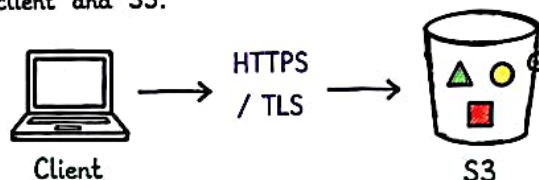
1. Encryption at Rest

Data is encrypted while it is stored in S3.

- **SSE-S3 (AES-256)** - Uses S3 managed keys.
- **SSE-KMS** - Uses AWS KMS keys. More control, audit and key rotation.
- **DSSE-KMS** - Dual layer encryption using two KMS keys (extra security).
- **Client-Side Encryption** - Encrypts data before upload using your own keys.

2. Encryption in Transit

Data is encrypted while it is moving between client and S3.

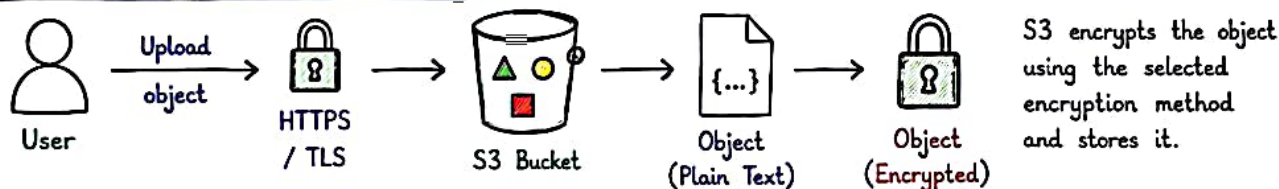


- Always use HTTPS/TLS to protect data in transit.
- Prevents eavesdropping and man-in-the-middle attacks.

3. SSE-S3 vs SSE-KMS

Feature	SSE-S3 (AES-256)	SSE-KMS
Key Management	Managed by AWS S3	Managed by AWS KMS
Control	Basic	Advanced (Key policies, IAM, Rotation)
Audit	Limited	Full (CloudTrail logs)
Cost	No extra cost	Additional KMS charges
Best For	Simple use cases	Compliance, control, audit requirements

4. How Encryption Works (At Rest)



5. Which Encryption Should You Use?

- SSE-S3** → For simple use cases where you want AWS to manage encryption for you.
- SSE-KMS** → For better control, audit, compliance and key management.
- DSSE-KMS** → For highest security (dual layer encryption).
- Client-Side Encryption** → When you want full control and AWS should never see your keys.

6. Example Use Cases

- Static website files, images, logs → **SSE-S3**
- Business data, backups, reports → **SSE-KMS**
- Sensitive / regulated data (HIPAA, PCI, etc.) → **SSE-KMS**
- High security requirement (extra protection) → **DSSE-KMS**
- Data where AWS should not manage keys → **Client-Side Encryption**

7. Key Points

- Data in S3 is encrypted by default (SSE-S3).
- You can choose stronger encryption (SSE-KMS or DSSE-KMS) based on your security and compliance needs.
- Always use HTTPS/TLS for data in transit.
- Manage KMS keys properly with IAM policies.
- Monitor access using CloudTrail and S3 Access Logs.



Encrypt your data. Protect your business. Stay secure.



S3 PRICING & COST

13

S3 is a pay-as-you-go service. You pay only for what you use. Many factors affect the final cost like storage, requests, data transfer and extra features.

1. What You Pay For?

You are charged for:

- Storage (based on storage class and region)
- Requests (PUT, GET, LIST, DELETE, etc.)
- Data transfer (out of S3 / internet)
- Data transfer (between AWS services / regions)
- Additional features (replication, lifecycle, inventory, Glacier, etc.)

2. Storage Pricing (Main Factor)

You pay based on:

- ✓ Amount of data stored
- ✓ Storage class you choose
- ✓ Region where bucket is located



Different storage class
=
Different cost

3. S3 Storage Classes (Cost Comparison)

Storage Class	Best For	Durability	Availability	Min. Storage Duration	Cost
Standard	Frequently accessed data	99.999999999%	99.99%	-	\$\$\$ (Highest)
Intelligent-Tiering	Data with unknown access pattern	99.999999999%	99.9%	-	\$\$ (Auto-optimizes)
Standard-IA	Infrequently accessed data	99.999999999%	99.9%	30 days	\$ (Lower)
One Zone-IA	Recreateable, less important data	99.999999999%	99.5%	30 days	\$ (Lower)
Glacier Instant Retrieval	Archive data, need millisecond access	99.999999999%	99.9%	90 days	\$ (Low)
Glacier Flexible Retrieval	Archive data, minutes or hours access	99.999999999%	99.9%	90 days	\$ (Very Low)
Glacier Deep Archive	Long term archive data, lowest cost	99.999999999%	99.9%	180 days	\$ (Lowest)

4. Request Pricing (Examples)

- PUT, COPY, POST, LIST requests → Charged per 1,000 requests
- GET, SELECT requests → Charged per 1,000
- DELETE requests → Charged per 1,000
- Lifecycle transitions (moves between storage classes) → Charged per 1,000 objects

5. Data Transfer Pricing

- Data OUT to internet → Charged per GB
- Data OUT to another region → Charged per GB
- Data OUT to AWS services in same region → Free
- Data IN to S3 → Free

6. Cost Optimization Tips

- ✓ Choose the right storage class for your data.
- ✓ Use Lifecycle rules to move data to cheaper storage automatically.
- ✓ Delete incomplete multipart uploads.
- ✓ Use S3 Intelligent-Tiering for unknown access pattern.
- ✓ Enable S3 Transfer Acceleration only if needed.
- ✓ Monitor usage with AWS Cost Explorer and S3 Storage Lens.



7. Example Cost Factors

- 100 GB in Standard = Higher cost
- 100 GB in Glacier Deep Archive = Much lower cost
- 1M GET requests = More cost
- Data transfer 100 GB out to internet = Extra cost



Check the latest pricing on AWS Pricing Calculator as prices may change by region and time.



Pay only for what you use!



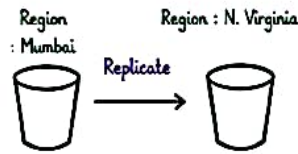
S3 ADVANCED FEATURES

14

S3 has many advanced features that help you manage data efficiently, improve performance, ensure compliance and build event-driven applications.

1. Cross-Region Replication (CRR)

- Automatically copies objects to a bucket in another AWS Region.
- Useful for disaster recovery, compliance and low latency access.



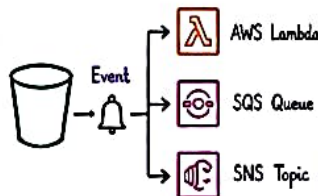
2. Transfer Acceleration

- Uses AWS edge locations to speed up data transfer to S3.
- Useful for users far from the AWS Region.



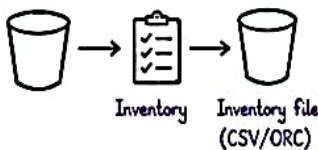
3. S3 Event Notifications

- Get alerts when events happen in your bucket.
- Events: Object Created, Deleted, Restore, Lifecycle transitions, etc.



4. S3 Inventory

- Provides a daily or weekly list of objects in your bucket.
- Helps with auditing, reporting and compliance.



8. Other Useful Features

Feature	What it does	Use case
S3 Batch Operations	Perform bulk actions on objects (copy, delete, restore, tag, etc.)	Manage millions of objects easily
Access Points	Create unique access points for shared datasets in a bucket	Simplify data access for multiple users or teams
Object Lambda	Run Lambda functions to customize data before it is returned	Modify or filter data on the fly
AWS Glue Integration	Catalog and analyze S3 data for analytics	Big data, ETL and data lake use cases
S3 on Outposts	Run S3 on your on-premises environment	Low latency, data residency

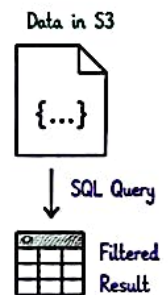
5. Object Lock (WORM)

- Write Once Read Many - objects cannot be overwritten or deleted for a fixed time.
- Helps meet compliance requirements.



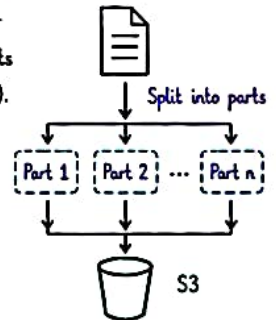
6. S3 Select

- Query data directly from S3 using SQL.
- Returns only the data you need (no need to download full file).



7. Multi-Part Upload

- Uploads large files in parts (5 MB to 5 TB per part).
- Faster, more reliable and can resume on failure.



9. S3 Naming Rules (Quick)

- Bucket name must be globally unique.
- 3 to 63 characters long.
- Lowercase letters, numbers, hyphens (-) and dots (.) only.
- Must start and end with a letter or number.
- No IP address format (e.g. 192.168.1.1).
- Avoid "xn--" prefix and "s3" suffix.



Choose a name that is short, descriptive and future-proof.

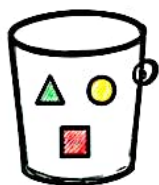
10. Key Points

- ✓ Use CRR for backup and disaster recovery.
- ✓ Enable Transfer Acceleration for faster uploads/downloads globally.
- ✓ Use lifecycle rules and inventory for cost optimization.
- ✓ Use Object Lock for compliance and data immutability.
- ✓ Use event notifications to build event-driven architectures.
- ✓ Always choose the right storage class and feature for your use case.



Powerful features. Smart storage. Better outcomes.





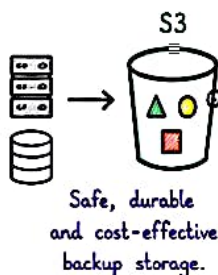
S3 REAL-WORLD USE CASES

15

Amazon S3 is used in many real-world scenarios to store, protect and deliver data at massive scale.

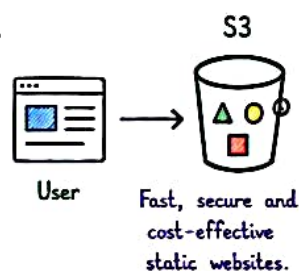
1. Backup & Archival

- Store backups of databases, servers, applications, files, etc.
- Use lifecycle rules to move old data to cheaper storage classes like Glacier.



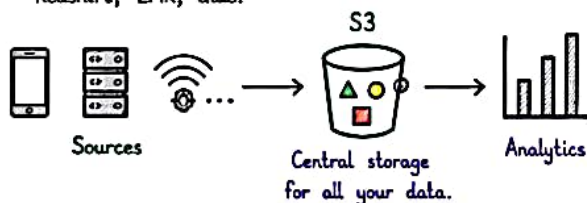
2. Static Website Hosting

- Host static websites (HTML, CSS, JS, images) directly on S3.
- Enable website hosting and access via a public endpoint or CloudFront.



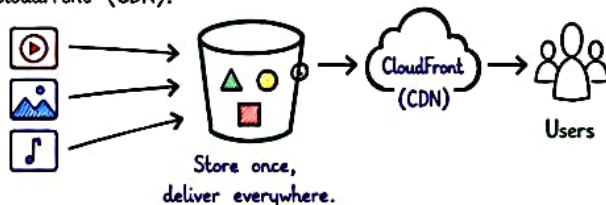
3. Data Lake / Analytics

- Store raw data from multiple sources (logs, app data, IoT, etc.)
- Analyze using services like Athena, Redshift, EMR, Glue.



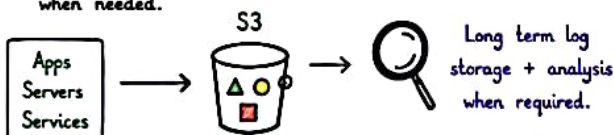
4. Media & Streaming

- Store images, videos, audio files for media apps.
- Deliver content globally using CloudFront (CDN).



5. Log Management

- Store application, server, and access logs.
- Use lifecycle to archive logs and analyze when needed.



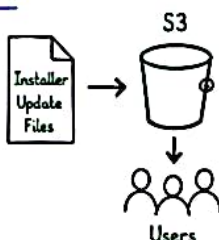
6. Disaster Recovery

- Replicate data across regions (CRR) for disaster recovery.
- Ensure business continuity with versioning and cross-region copies.



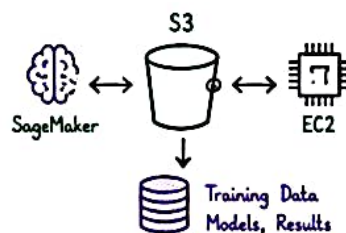
7. Software Delivery / Updates

- Store software installers, patches, and updates.
- Users can download from S3 or CloudFront.



8. ML / AI Workloads

- Store training data, models, and results.
- Integrate with SageMaker, EC2, and other ML services.



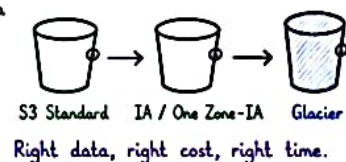
9. Compliance & Legal Hold

- Use Object Lock (WORM) to prevent data deletion or modification.
- Useful for legal, financial and regulatory requirements.



10. Cost Optimization

- Use lifecycle rules to move data to lower-cost storage classes.
- Delete expired or unnecessary data automatically.

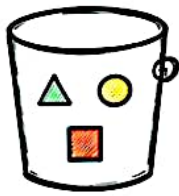


Why S3 is so powerful?

- ✓ Highly durable (11 9's) and available.
- ✓ Scalable to petabytes of data.
- ✓ Secure, flexible and cost-effective.
- ✓ Works with many AWS services.



From small projects to global applications, S3 is the **backbone** of modern cloud storage!



S3 BEST PRACTICES

16

Following best practices helps you build secure, reliable, and cost-effective S3 solutions.

1. Security Best Practices

- ✓ Use IAM roles and **least privilege** access.
- ✓ Block public access unless required.
- ✓ Use Bucket Policies instead of ACLs.
- ✓ Enable encryption (at rest & in transit).
- ✓ Enable versioning and MFA delete.
- ✓ Monitor with CloudTrail and S3 Access Logs.
- ✓ Scan objects with Macie for sensitive data.



2. Performance Best Practices

- Use S3 Transfer Acceleration for faster global uploads/downloads.
- Use CloudFront in front of S3 for caching and low latency.
- Use multipart upload for large files (better speed and reliability).
- Distribute data across prefixes (avoid hot prefixes).



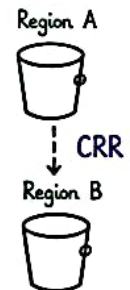
3. Cost Optimization Best Practices

- Use the right storage class for your data.
- Enable Lifecycle rules to move data to cheaper classes or delete when not needed.
- Delete incomplete multipart uploads.
- Use S3 Intelligent-Tiering for unknown or changing access patterns.

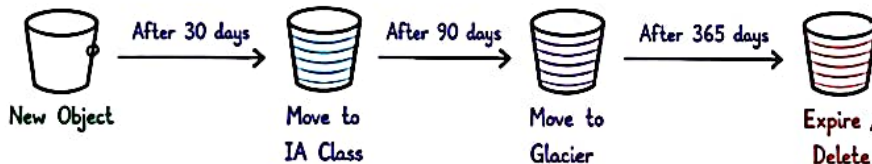


4. Reliability & Availability Best Practices

- Enable **Cross-Region Replication (CRR)** for disaster recovery.
- Enable versioning to protect against accidental delete/overwrite.
- Store important data in multiple AZs (default) and Regions (CRR).
- Test restore process regularly.



5. S3 Lifecycle Example



💡 Lifecycle =
Right data,
Right class,
Right time,
Lower cost.

6. Do's and Don'ts

Do's	Don'ts
✓ Use meaningful bucket names.	✗ Don't make buckets public unless required.
✓ Enable versioning.	✗ Don't store unnecessary data.
✓ Use encryption.	✗ Don't ignore encryption.
✓ Apply least privilege access.	✗ Don't share root access keys.
✓ Use lifecycle rules.	✗ Don't forget to clean up old/unused data.
✓ Monitor with CloudTrail & S3 logs.	✗ Don't ignore costs.

7. Monitoring & Auditing



8. Naming Convention Tips

- Use lowercase letters, numbers, hyphens (-).
- No underscores (_), spaces or uppercase.
- 3 to 63 characters long.
- Start and end with letter or number.
- Example: my-company-backup-prod-logs

Keep it
simple,
clear and
unique!

9. Key Takeaways

- S3 is highly durable, scalable and secure when used right.
- Follow best practices for security, performance, cost and reliability.
- Always review and optimize your S3 setup.



Secure it well.
Optimize it smartly.
Scale it globally.

Build better
with AWS S3!



Best practices today → Reliable, Secure & Cost-effective S3 tomorrow!